

JAVA DOCS

Object-Oriented Programming in Java

Classes, Inheritance, Polymorphism, Abstraction & Encapsulation

Java Agentic AI — Knowledge Base Reference

1. The Four Pillars of OOP

Java is built around object-oriented programming, organized around four foundational principles: **Encapsulation**, **Inheritance**, **Polymorphism**, and **Abstraction**. Together they let programs model real-world entities as objects that bundle state (fields) and behavior (methods).

Pillar	One-line definition
Encapsulation	Bundling data and methods together, restricting direct access to internal state
Inheritance	A class acquiring fields/methods of another class, enabling code reuse
Polymorphism	One interface, many implementations — the same call behaves differently by context
Abstraction	Exposing only essential features, hiding implementation complexity

2. Classes and Objects

A **class** is a blueprint defining fields (state) and methods (behavior). An **object** is a concrete instance of a class, created with the `new` keyword, living on the heap.

```
public class Car {
    // fields (instance state)
    private String model;
    private int speed;

    // constructor
    public Car(String model) {
        this.model = model;
        this.speed = 0;
    }

    // methods (behavior)
    public void accelerate(int amount) {
        this.speed += amount;
    }

    public int getSpeed() {
        return speed;
    }
}

Car myCar = new Car("Tesla Model 3"); // object creation
myCar.accelerate(50);
System.out.println(myCar.getSpeed()); // 50
```

2.1 Constructors

- A constructor has the same name as the class and no return type.
- If no constructor is defined, the compiler supplies a no-arg **default constructor**.
- **Constructor overloading**: multiple constructors with different parameter lists.
- `this(...)` calls another constructor in the same class (must be the first statement).

- `super(...)` calls the parent class constructor (must be the first statement).

```
public class Car {
    private String model;
    private int speed;

    public Car() {
        this("Unknown Model");    // constructor chaining via this()
    }

    public Car(String model) {
        this.model = model;
        this.speed = 0;
    }
}
```

3. Encapsulation

Encapsulation means declaring fields private and exposing controlled access through public getters/setters (or immutable design with no setters). This protects invariants and lets the internal representation change without breaking client code.

```
public class BankAccount {
    private double balance;    // hidden internal state

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > balance) throw new IllegalStateException("Insufficient funds");
        balance -= amount;
    }
}
```

Note: Without encapsulation, any code could do `account.balance = -9999` directly. With it, every mutation is forced through validated methods.

4. Inheritance

Inheritance lets a subclass (child) acquire fields and methods from a superclass (parent) using `extends`. Java supports only **single inheritance** for classes (a class extends exactly one other class), but a class may implement multiple interfaces.

```
public class Animal {
    protected String name;

    public Animal(String name) { this.name = name; }
```

```

        public void eat() { System.out.println(name + " is eating"); }
    }

    public class Dog extends Animal {
        public Dog(String name) {
            super(name);          // must call superclass constructor first
        }

        public void bark() { System.out.println(name + " says Woof!"); }
    }

    Dog d = new Dog("Rex");
    d.eat();      // inherited from Animal
    d.bark();     // defined in Dog

```

4.1 IS-A vs HAS-A

- **IS-A** (inheritance): Dog IS-A Animal — use extends/implements.
- **HAS-A** (composition): Car HAS-A Engine — use a field reference instead of inheritance. Favored in modern design ('favor composition over inheritance') because it avoids tight coupling and fragile base class problems.

4.2 The Object Class

Every class implicitly extends `java.lang.Object` if it extends nothing else. `Object` provides `equals()`, `hashCode()`, `toString()`, `getClass()`, `clone()`, and the wait/notify family used for thread coordination.

5. Polymorphism

5.1 Compile-Time (Static) Polymorphism — Overloading

Multiple methods share a name but differ in parameter type/count/order. Resolved by the compiler based on the declared argument types.

```

class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}

```

5.2 Runtime (Dynamic) Polymorphism — Overriding

A subclass provides a specific implementation of a method already defined in its superclass, using the `@Override` annotation. The actual method invoked is determined at runtime based on the object's real (runtime) type, not its declared (reference) type — this is called **dynamic method dispatch**.

```

class Animal {
    void speak() { System.out.println("Some generic sound"); }
}

class Cat extends Animal {

```

```

    @Override
    void speak() { System.out.println("Meow"); }
}

Animal a = new Cat();    // reference type Animal, runtime type Cat
a.speak();               // prints "Meow" - dynamic dispatch uses runtime type

```

5.3 Overloading vs Overriding

Aspect	Overloading	Overriding
Where	Same class (or subclass with different signature)	Subclass redefines a superclass method
Signature	Must differ (params)	Must be identical (name + params)
Binding	Compile-time (static)	Runtime (dynamic)
Return type	Can differ freely	Must be same or a covariant subtype
Access modifier	No restriction	Cannot reduce visibility

6. Abstraction

6.1 Abstract Classes

An abstract class cannot be instantiated directly and may contain both abstract methods (no body, must be implemented by subclasses) and concrete methods (shared implementation).

```

public abstract class Shape {
    abstract double area();           // no body - subclass must implement

    void describe() {                 // concrete method - shared by all subclasses
        System.out.println("Area = " + area());
    }
}

public class Circle extends Shape {
    private double radius;
    Circle(double radius) { this.radius = radius; }

    @Override
    double area() { return Math.PI * radius * radius; }
}

```

6.2 Interfaces

An interface defines a contract of behavior. Since Java 8, interfaces can include default methods (with a body, inherited unless overridden), static methods, and (Java 9+) private helper methods. All fields in an interface are implicitly public static final.

```

public interface Flyable {
    void fly();                       // implicitly public abstract
}

```

```

    default void land() {                                // default method
        System.out.println("Landing safely");
    }

    static Flyable createDefault() {                    // static factory method
        return () -> System.out.println("Generic flying");
    }
}

public class Bird implements Flyable {
    @Override
    public void fly() { System.out.println("Bird flying"); }
}

```

6.3 Abstract Class vs Interface

Aspect	Abstract Class	Interface
Inheritance	extends — single only	implements — multiple allowed
Constructors	Can have one	Cannot have one
Fields	Any type (instance state allowed)	Only public static final constants
Method bodies	Any mix of abstract/concrete	default/static/private methods have bodies
Use case	Shared base state + partial implementation	Pure capability contract, multiple inheritance of type

Note: Since Java 8 blurred many practical differences via default methods, a common guideline is: use an abstract class when subclasses share common state/implementation; use an interface to define a capability that unrelated classes can implement (e.g. Comparable, Runnable, Serializable).

7. this and super Keywords

Keyword	Refers to
this	The current object instance; disambiguates fields from parameters; calls another constructor
super	The immediate parent class; accesses parent fields/methods hidden by overriding; calls parent constructor

```

class Base {
    int x = 10;
    void show() { System.out.println("Base.show"); }
}
class Derived extends Base {
    int x = 20;
    void show() {
        System.out.println(super.x); // 10 - parent's field
        System.out.println(this.x);  // 20 - this object's field
        super.show();                // calls Base's show()
    }
}

```

Note: Field access is resolved statically by declared type (no polymorphism for fields), which is why field 'hiding' behaves differently from method overriding — this is a frequent interview trap.

8. equals(), hashCode() and toString()

```
public class Point {
    private final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }

    @Override
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
}
```

Note: The equals()/hashCode() contract: if two objects are equal, they **MUST** have the same hashCode. Violating this breaks HashMap/HashSet lookups silently (an object can appear 'lost' even though it was inserted, because it lands in a different bucket than an equal lookup key).

9. Object Initialization Order

- 1. Static fields and static initializer blocks of the class (once, at class-load time, top-to-bottom).
- 2. When 'new' is called: parent class's static init (if not already loaded), then instance fields and instance initializer blocks, then the constructor body — done for the superclass chain first, then the subclass.
- 3. Order for a subclass instance: Parent static -> Child static -> Parent instance fields/blocks -> Parent constructor -> Child instance fields/blocks -> Child constructor.

9b. Nested and Inner Classes

Type	Description
Static nested class	Declared static inside another class; behaves like a top-level class, no implicit outer reference
Inner (non-static) class	Holds an implicit reference to its enclosing instance; needs an outer object to be created
Local class	Defined inside a method body; can access effectively-final local variables
Anonymous class	One-off unnamed class, typically implementing an interface or extending a class inline

```

public class Outer {
    private int value = 42;

    class Inner {                                // non-static inner class
        void show() {
            System.out.println("Outer value: " + value); // accesses outer instance
directly
        }
    }

    static class StaticNested {                  // static nested class
        void show() { System.out.println("No outer instance needed"); }
    }
}

Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();          // requires an Outer instance
inner.show();

Outer.StaticNested nested = new Outer.StaticNested();

// Anonymous class implementing Runnable
Runnable r = new Runnable() {
    @Override
    public void run() { System.out.println("Running anonymously"); }
};
r.run();

```

9c. Composition Example (HAS-A)

Composition builds complex objects out of simpler ones held as fields, favoring flexibility over the rigid coupling of inheritance.

```

class Engine {
    void start() { System.out.println("Engine starting"); }
}

class Car {
    private final Engine engine;    // HAS-A relationship

    Car(Engine engine) { this.engine = engine; }

    void start() {
        engine.start();             // delegate to the composed object
        System.out.println("Car ready to drive");
    }
}

Car car = new Car(new Engine());
car.start();

```

Note: Composition allows swapping the Engine implementation (e.g. *ElectricEngine* vs *CombustionEngine*) at runtime without changing Car's class hierarchy — this flexibility is why 'favor composition over inheritance' is a widely repeated design guideline.

10. Quick Interview Recap

- Can a constructor be private? Yes — used in Singleton pattern and static factory methods.
- Can we override a static method? No — static methods are hidden, not overridden (resolved by reference type at compile time, not runtime polymorphism).
- Can an abstract class have a constructor? Yes — invoked via `super()` when a subclass is instantiated, even though the abstract class itself can never be instantiated directly.
- Why can't Java have multiple class inheritance? To avoid the 'diamond problem' ambiguity when two parent classes define conflicting method implementations; interfaces avoid this via explicit default-method conflict resolution rules.
- Difference between method hiding and overriding? Hiding applies to static methods/fields (compile-time, reference-type-based); overriding applies to instance methods (runtime, object-type-based).